

Automation: Telegram Quick Notes → AI-Expanded Notion Entries

Send a 2-3 word note to a Telegram bot. n8n catches it, sends it to Claude Haiku for expansion, and saves a structured entry into Notion. You get a confirmation back on Telegram.

Architecture

Telegram message → n8n Telegram Trigger → HTTP Request (Claude API) → Code node (parse JSON) → Notion node → Telegram confirmation

Step 1: Create the Telegram Bot

1. Open Telegram, search for **@BotFather**
2. Send **/newbot**
3. Give it a name (e.g. "Idea Vault Bot") and a username (must end in **bot** , e.g. **ayman_ideas_bot**)
4. BotFather gives you a **bot token** — looks like **123456789:AAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx**
5. Save this token, you'll need it in n8n
6. Open a chat with your new bot and send **/start** (bots can't message you first)



BotFather

8 082 821 monthly users



What can this bot do?

BotFather is the one bot to rule them all. Use it to create new bot accounts and manage your existing bots.

About Telegram bots:

<https://core.telegram.org/bots>

Bot API manual:

<https://core.telegram.org/bots/api>

Contact @BotSupport if you have questions about the Bot API.

START



BotFather

8 082 821 monthly users



Today

/start 08:29 PM ✓

I can help you create and manage Telegram bots. If you're new to the Bot API, please [see the manual](#).

You can control me by sending these commands:

[/newbot](#) - create a new bot
[/mybots](#) - edit your bots

Edit Bots

[/setname](#) - change a bot's name
[/setdescription](#) - change bot description
[/setabouttext](#) - change bot about info
[/setuserpic](#) - change bot profile photo
[/setcommands](#) - change the list of commands
[/deletebot](#) - delete a bot

Bot Settings

[/token](#) - get authorization token
[/revoke](#) - revoke bot access token
[/setinline](#) - toggle inline mode
[/setinlinegeo](#) - toggle inline location requests
[/setinlinefeedback](#) - change inline feedback settings
[/setjoingroups](#) - can your bot be added to groups?
[/setprivacy](#) - toggle privacy mode in groups

Web Apps

[/myapps](#) - edit your web apps
[/newapp](#) - create a new web app
[/listapps](#) - get a list of your web apps
[/editapp](#) - edit a web app
[/deleteapp](#) - delete an existing web app

Games

[/mygames](#) - edit your games
[/newgame](#) - create a new game
[/listgames](#) - get a list of your games
[/editgame](#) - edit a game
[/deletegame](#) - delete an existing game

08:29 PM



Open



Message





BotFather

8 082 821 monthly users



Today

/start

08:29 PM ✓

I can help you create and manage Telegram bots. If you're new to the Bot API, please [see the manual](#).

You can control me by sending these commands:

[/newbot](#) - create a new bot
[/mybots](#) - edit your bots

Edit Bots

[/setname](#) - change a bot's name
[/setdescription](#) - change bot description
[/setabouttext](#) - change bot about info
[/setuserpic](#) - change bot profile photo
[/setcommands](#) - change the list of commands
[/deletebot](#) - delete a bot

Bot Settings

[/token](#) - get authorization token
[/revoke](#) - revoke bot access token
[/setinline](#) - toggle inline mode
[/setinlinegeo](#) - toggle inline location requests
[/setinlinefeedback](#) - change inline feedback settings
[/setjoingroups](#) - can your bot be added to groups?
[/setprivacy](#) - toggle privacy mode in groups

Web Apps

[/myapps](#) - edit your web apps
[/newapp](#) - create a new web app
[/listapps](#) - get a list of your web apps
[/editapp](#) - edit a web app
[/deleteapp](#) - delete an existing web app

Games

[/mygames](#) - edit your games
[/newgame](#) - create a new game
[/listgames](#) - get a list of your games
[/editgame](#) - edit a game
[/deletegame](#) - delete an existing game

08:29 PM



Open



Message



BotFather



What is BotFather?

BotFather is the one bot to rule them all. Use it to create new bot accounts and manage your existing bots. [Learn more >](#)

Create a New Bot



BotFather

8 082 821 monthly users



Today

/start

08:29 PM ✓

I can help you create and manage Telegram bots. If you're new to the Bot API, please [see the manual](#).

You can control me by sending these commands:

[/newbot](#) - create a new bot
[/mybots](#) - edit your bots

Edit Bots

[/setname](#) - change a bot's name
[/setdescription](#) - change bot description
[/setabouttext](#) - change bot about info
[/setuserpic](#) - change bot profile photo
[/setcommands](#) - change the list of commands
[/deletebot](#) - delete a bot

Bot Settings

[/token](#) - get authorization token
[/revoke](#) - revoke bot access token
[/setinline](#) - toggle inline mode
[/setinlinegeo](#) - toggle inline location requests
[/setinlinefeedback](#) - change inline feedback settings
[/setjoingroups](#) - can your bot be added to groups?
[/setprivacy](#) - toggle privacy mode in groups

Web Apps

[/myapps](#) - edit your web apps
[/newapp](#) - create a new web app
[/listapps](#) - get a list of your web apps
[/editapp](#) - edit a web app
[/deleteapp](#) - delete an existing web app

Games

[/mygames](#) - edit your games
[/newgame](#) - create a new game
[/listgames](#) - get a list of your games
[/editgame](#) - edit a game
[/deletegame](#) - delete an existing game

08:29 PM



Message



BotFather



New bot

Enter a name, description and username to create a new bot.

Speed Notes

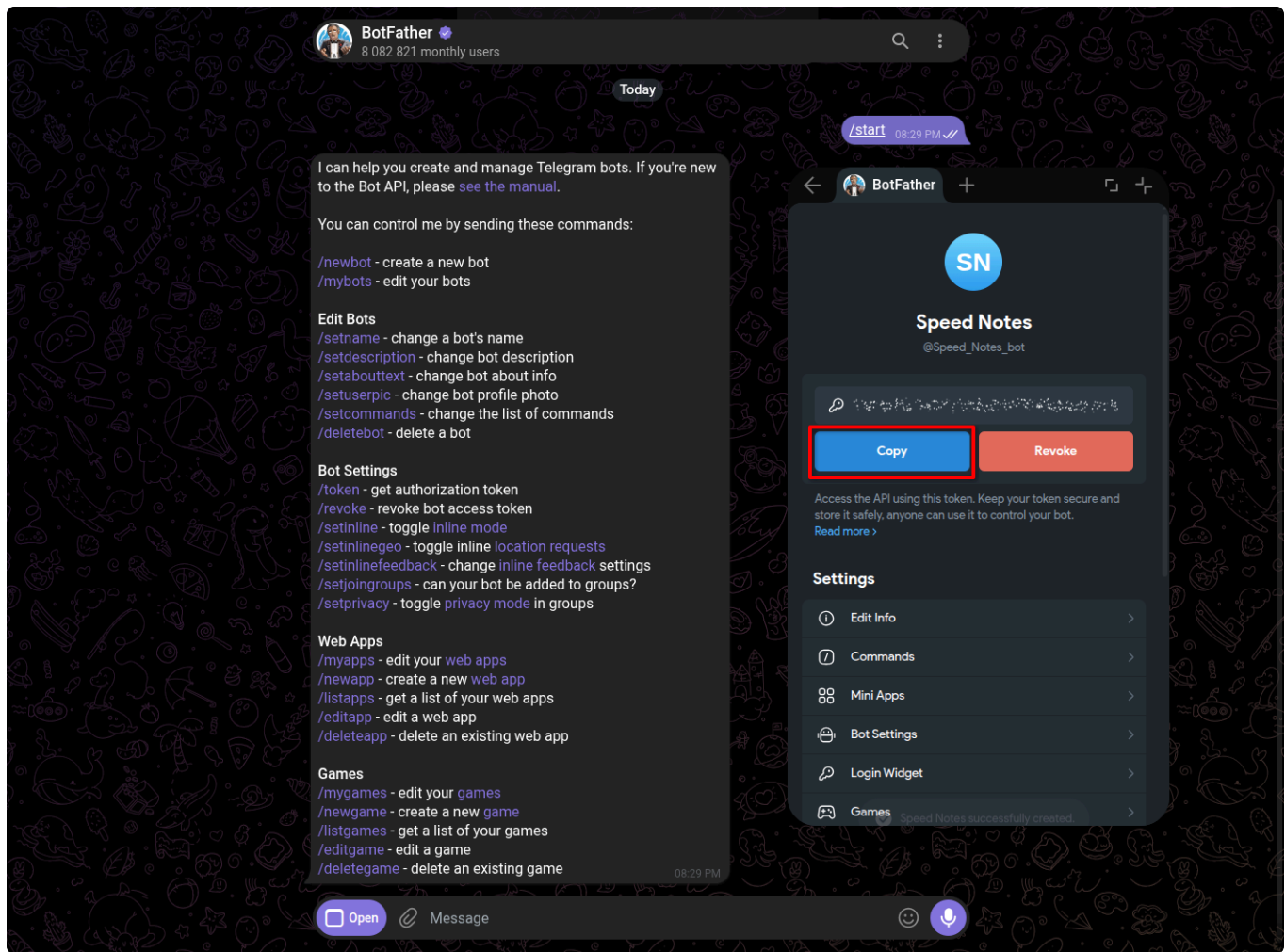
About (Optional)

t.me/Speed_Notes_bot

Speed_Notes_bot is available.

Choose a username for your bot. It must end in 'bot'. Like this, for example: TetrisBot or tetris_bot.

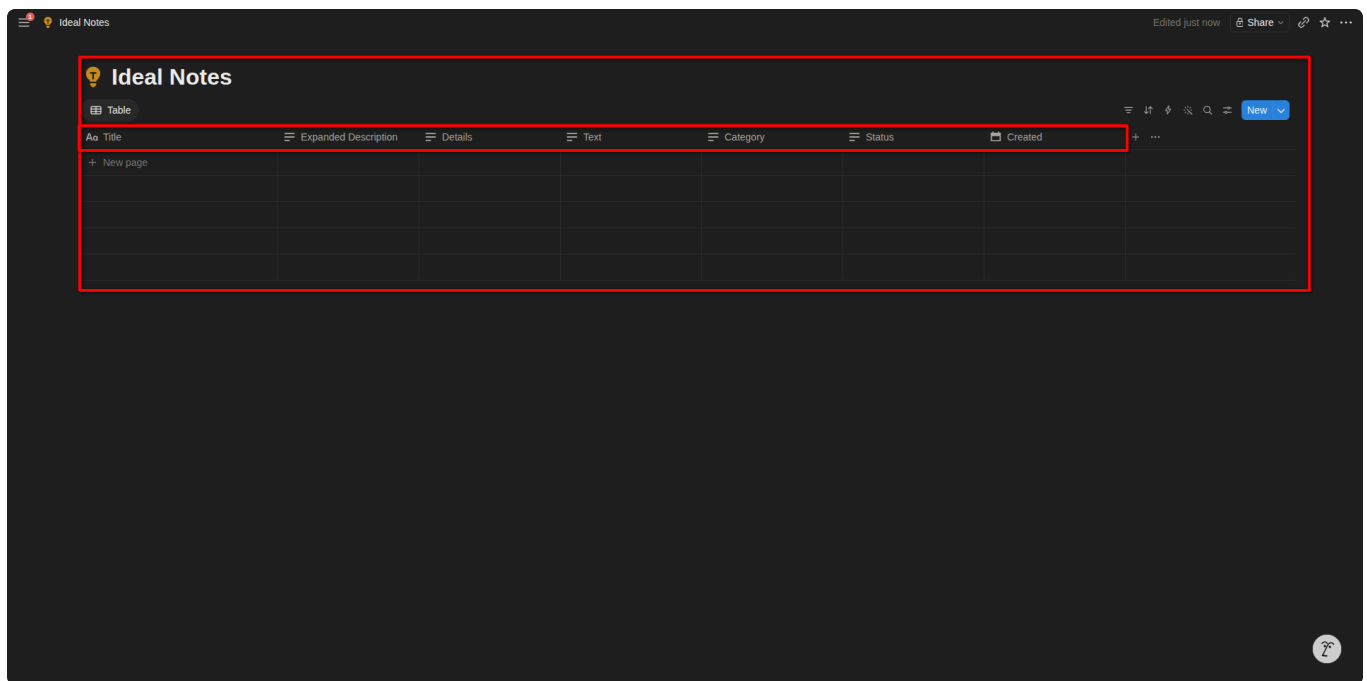
CREATE BOT



Step 2: Set Up the Notion Database

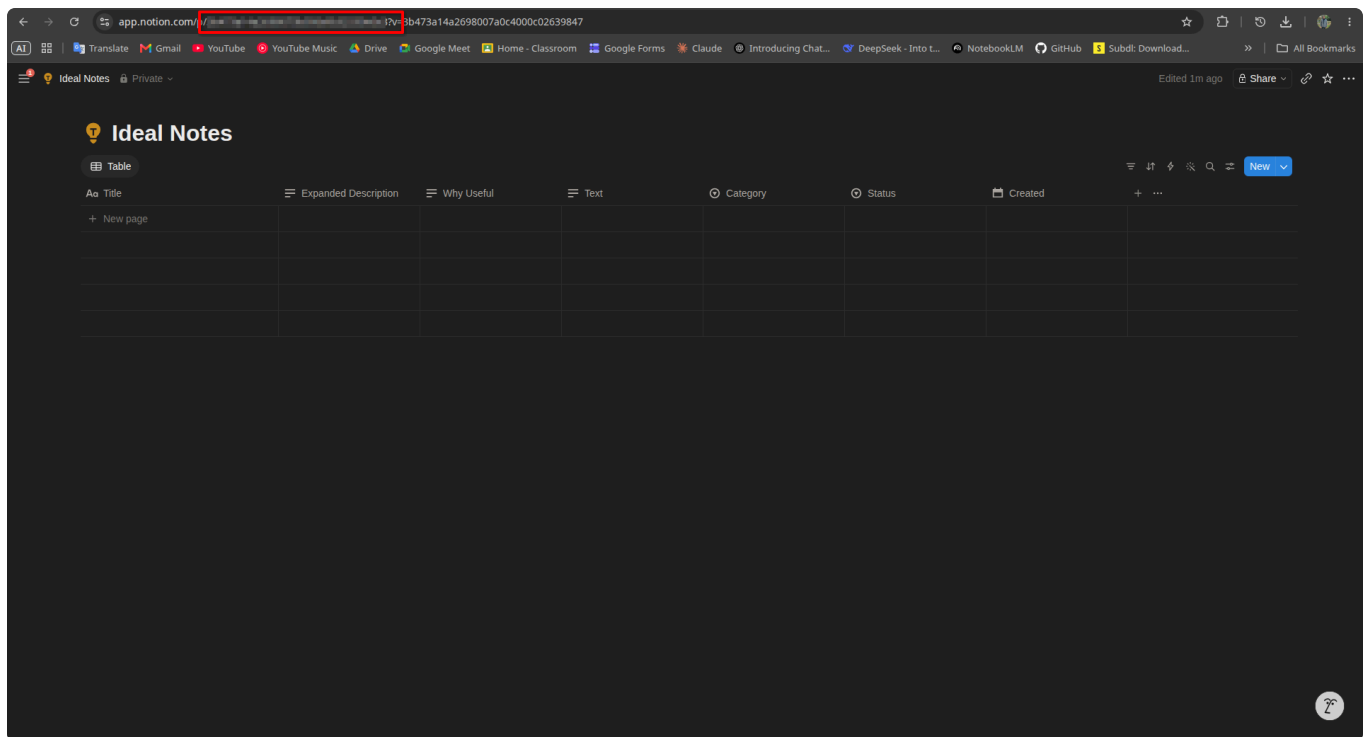
Create a new database called **Idea Vault** (or similar) with these properties:

Property	Type	Notes
Title	Title	Your original short note
Expanded Description	Rich Text	AI-generated description
Details	Rich Text	AI-generated relevance
Category	Select	Options: Feature Idea, Tool, Project, Random
Status	Select	Options: New, Reviewed, Actioned
Created	Date	Auto-set on creation



Get the Database ID:

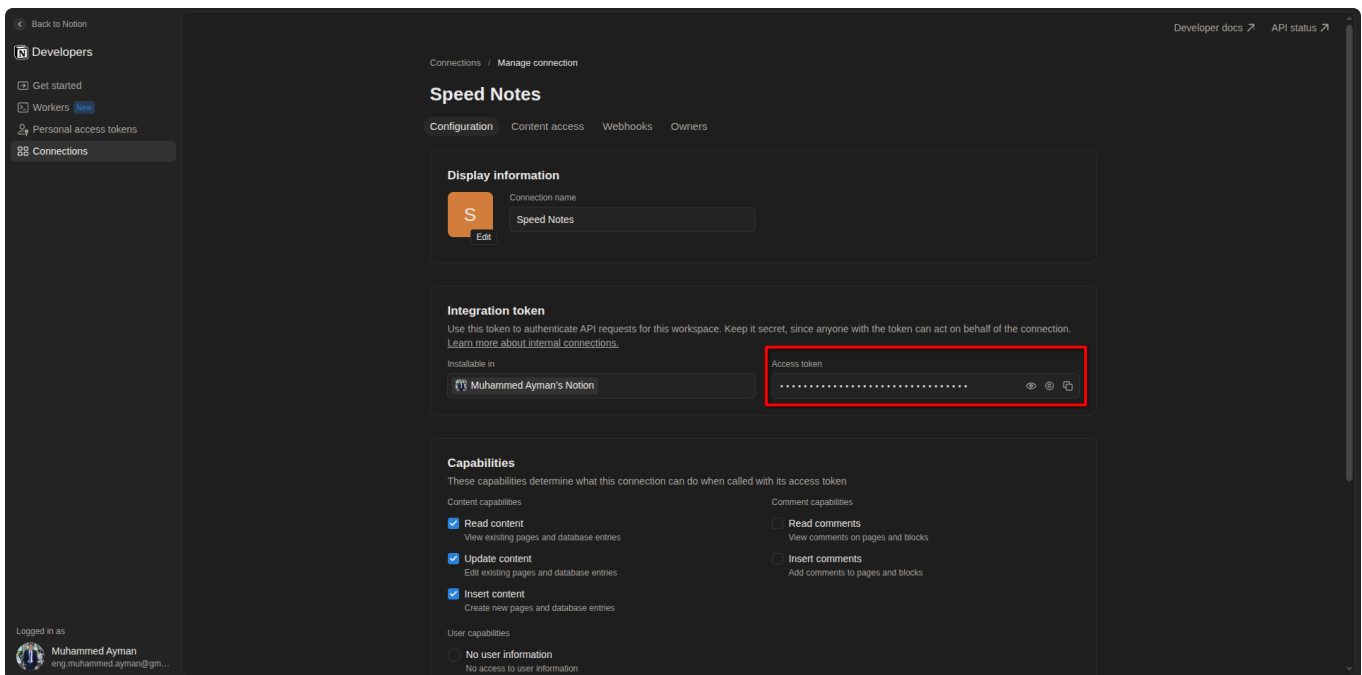
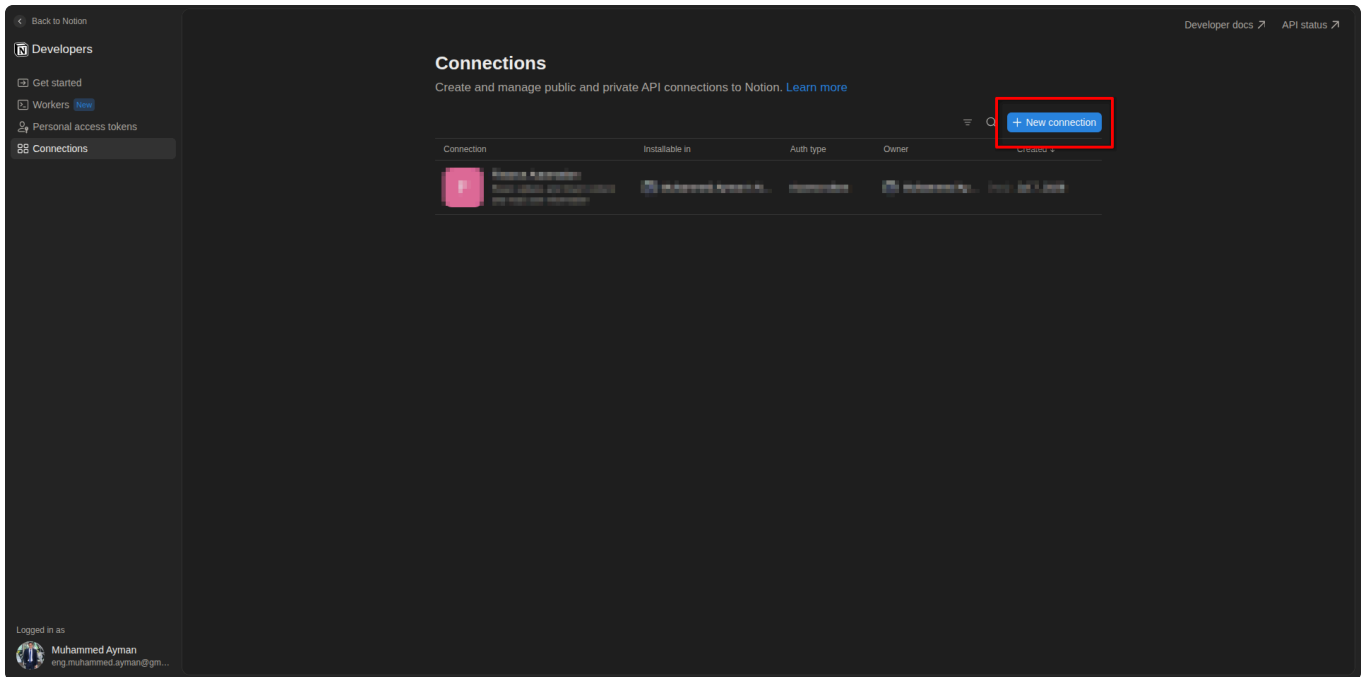
- Open the database as a full page
- Copy the URL: `https://notion.so/yourworkspace/DATABASE_ID?v=...`
- The 32-character string before `?v=` is your Database ID

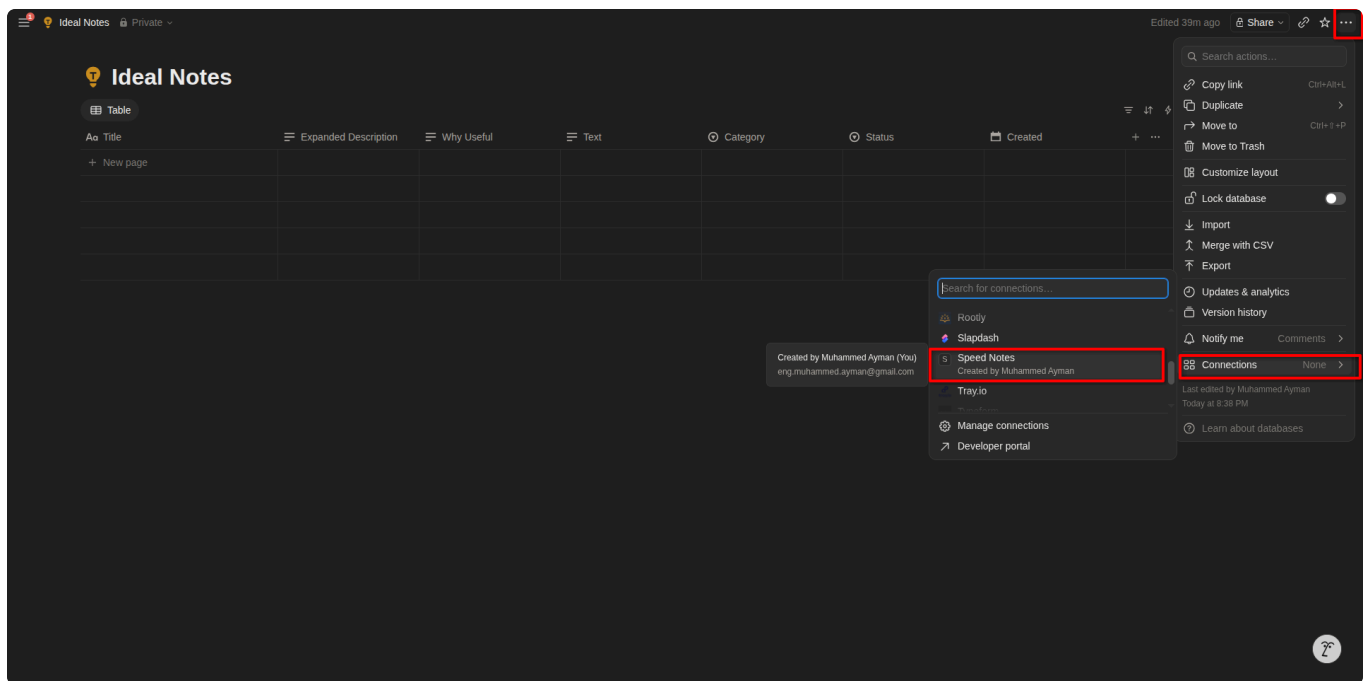


Connect Notion to n8n:

- In Notion, go to Settings → Connections → Develop or manage integrations → New integration (<https://app.notion.com/developers/connections>)

- Name it (e.g. "n8n automation"), copy the **Internal Integration Token**
- Go back to your Idea Vault database → ... menu → Connections → add your integration
- In n8n, add Notion credentials using that token





Step 3: Get a Free AI API Key (Google Gemini)

We'll use **Google AI Studio's Gemini API** — it has a generous free tier (no card required to start) and is plenty capable for expanding a short note into a structured description.

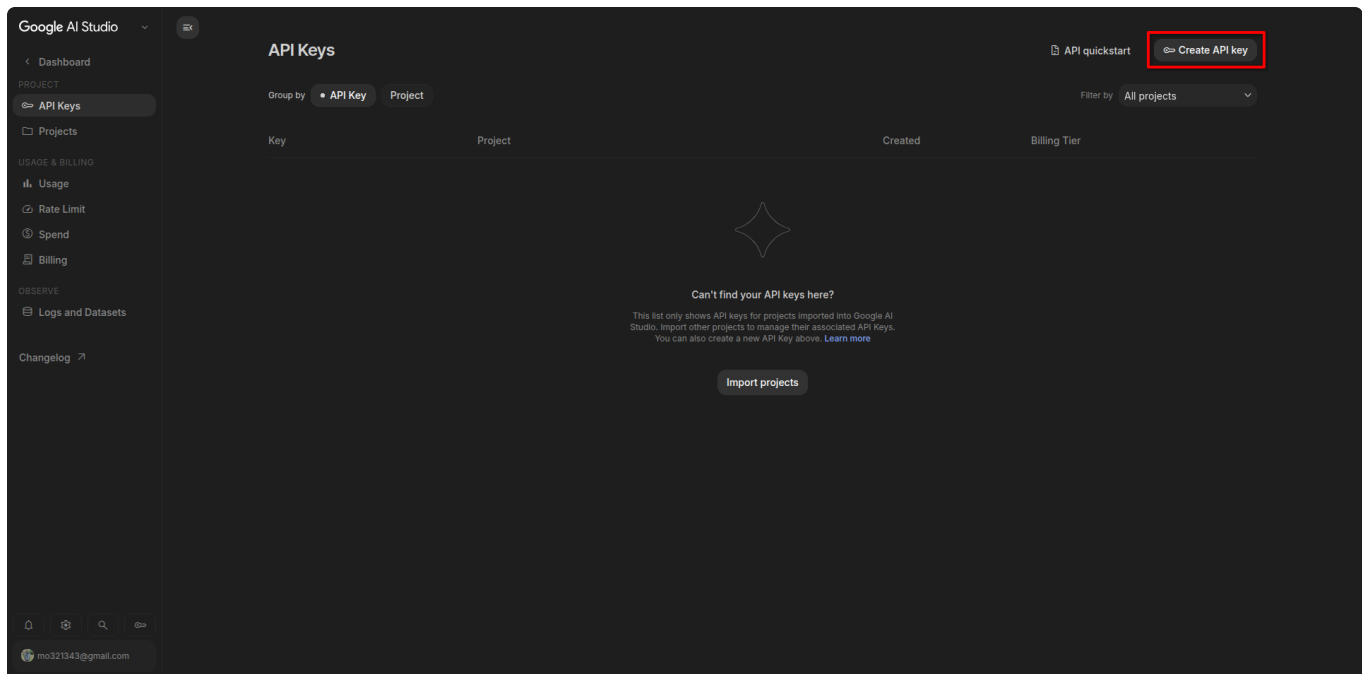
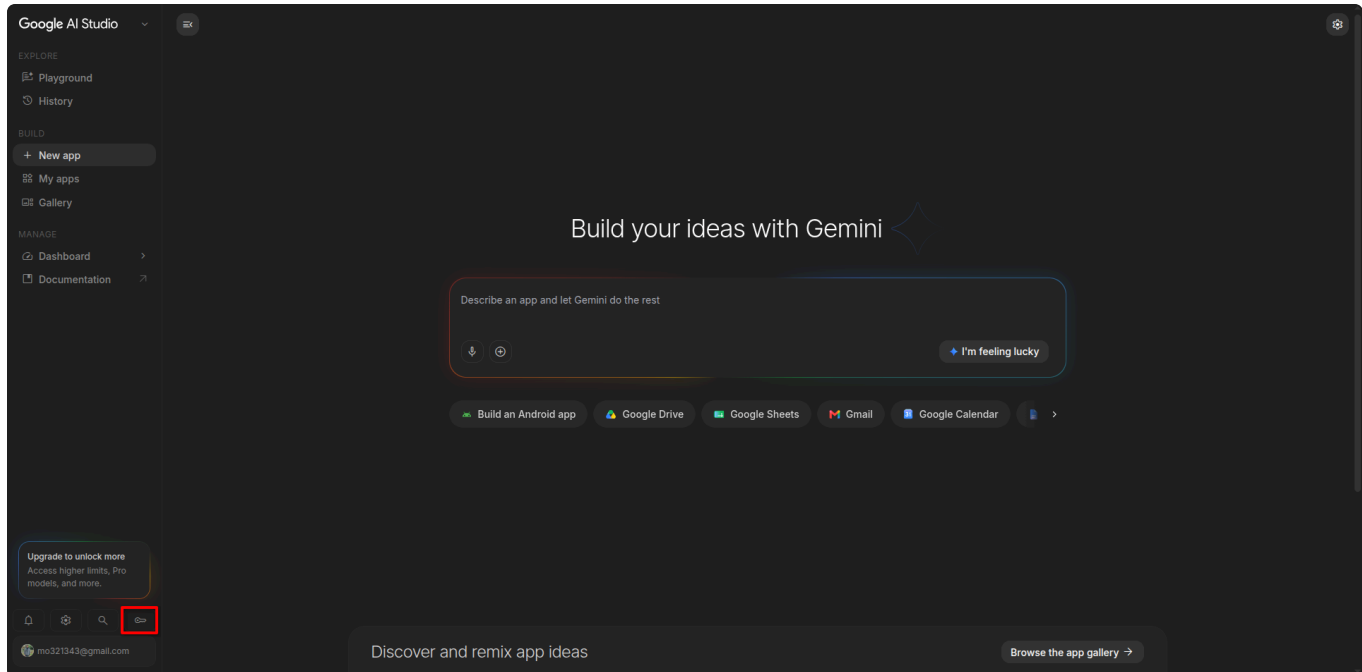
1. Go to aistudio.google.com
2. Sign in with your Google account
3. Click **Get API Key** (left sidebar) → **Create API Key**
4. Choose a Google Cloud project (or let it create one) and generate the key
5. Save the key — you'll paste it into the HTTP Request node in n8n

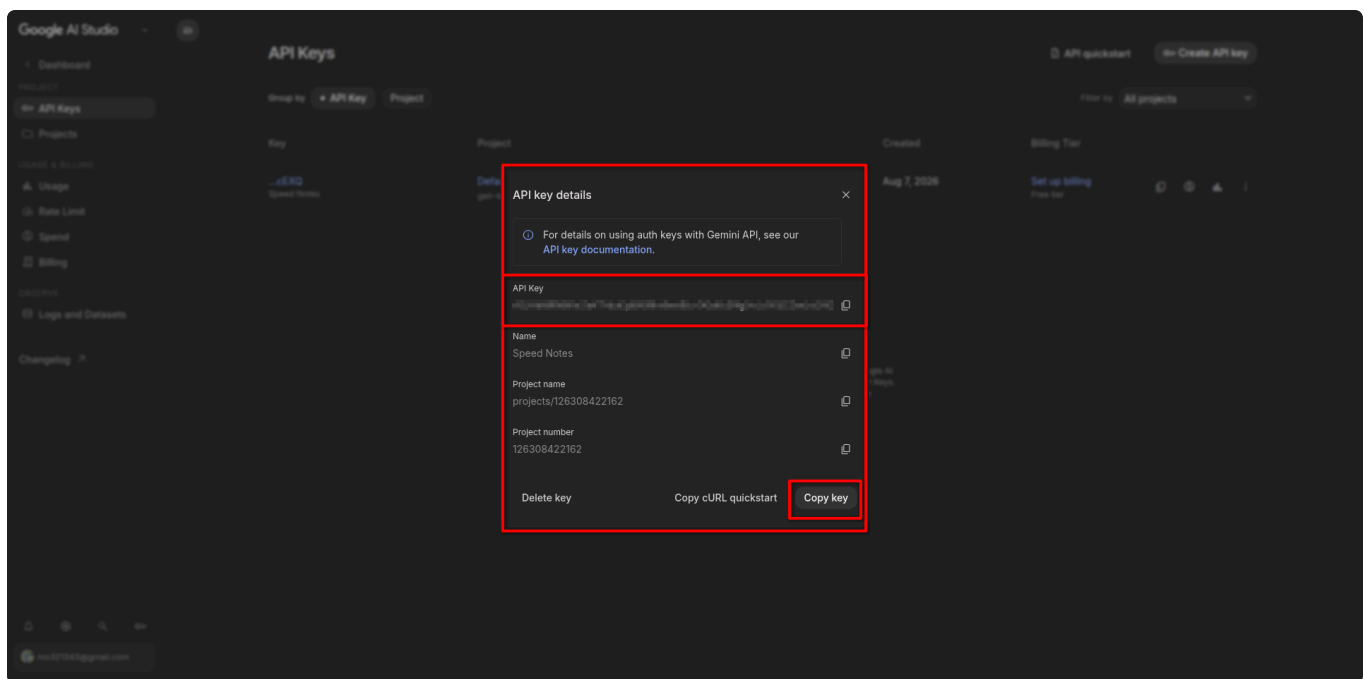
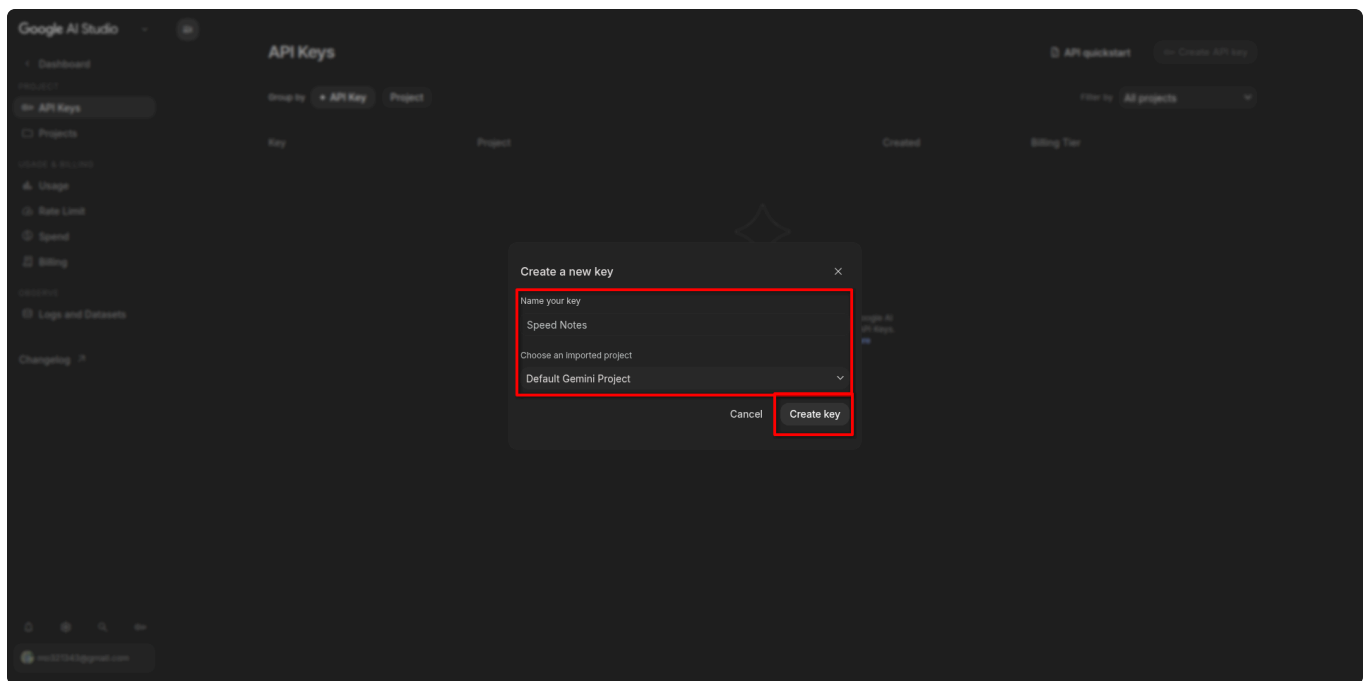
Free tier notes:

- Gemini's free tier has daily/per-minute request limits (varies by model). For a personal note-taking bot sending a handful of messages a day, you'll stay well within it.
- Use **gemini-3.6-flash** — this is the current stable, generally-available Flash model as of mid-2026. Google deprecates model versions fast (2.0 Flash and 2.5 Flash are both already retired for new users), so if this guide ever throws a "model not available" 404 again, check ai.google.dev/gemini-api/docs/models for whatever the current stable Flash model string is and swap it into the URL below.
- If you ever outgrow it, other free/cheap options worth knowing about: **Groq** (very fast inference, generous free tier) and **OpenRouter** (aggregates many models, some fully free).

If you get a **429 RESOURCE_EXHAUSTED / limit: 0 error**: This means your key is stuck in the most restrictive, unverified quota bucket — not that you've actually used up requests.
Fix:

1. Go to console.cloud.google.com/billing
2. Link a billing account to the same Google Cloud project your API key belongs to
3. Set a budget alert at \$0.01 so you're notified before any real charge (Gemini 2.5 Flash stays free under normal personal use even with billing linked)
4. Generate a fresh API key after linking, and retry

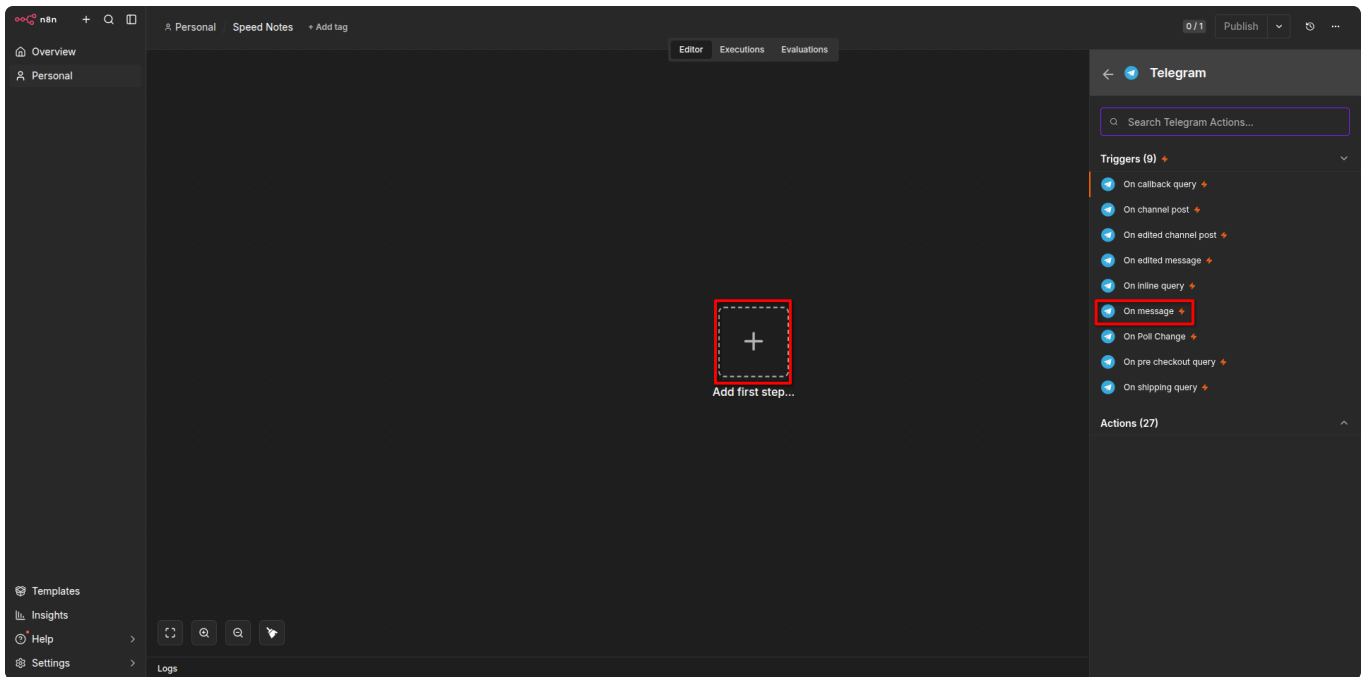
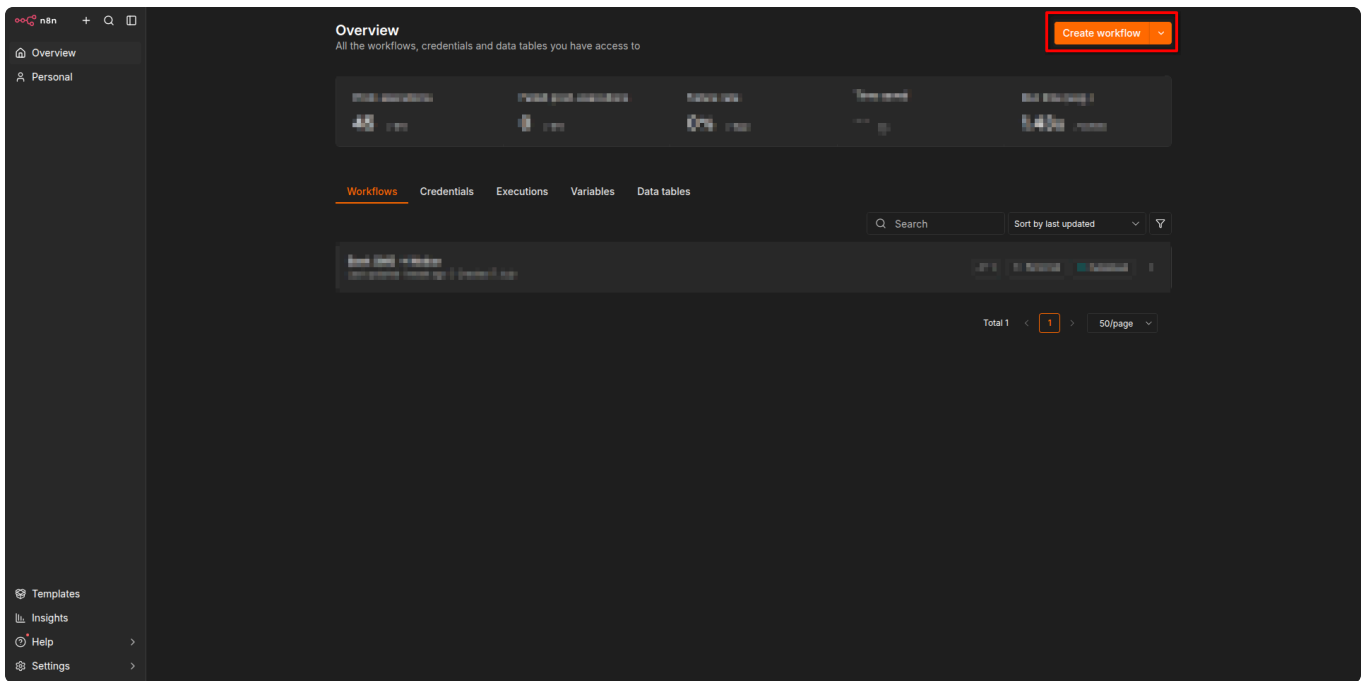


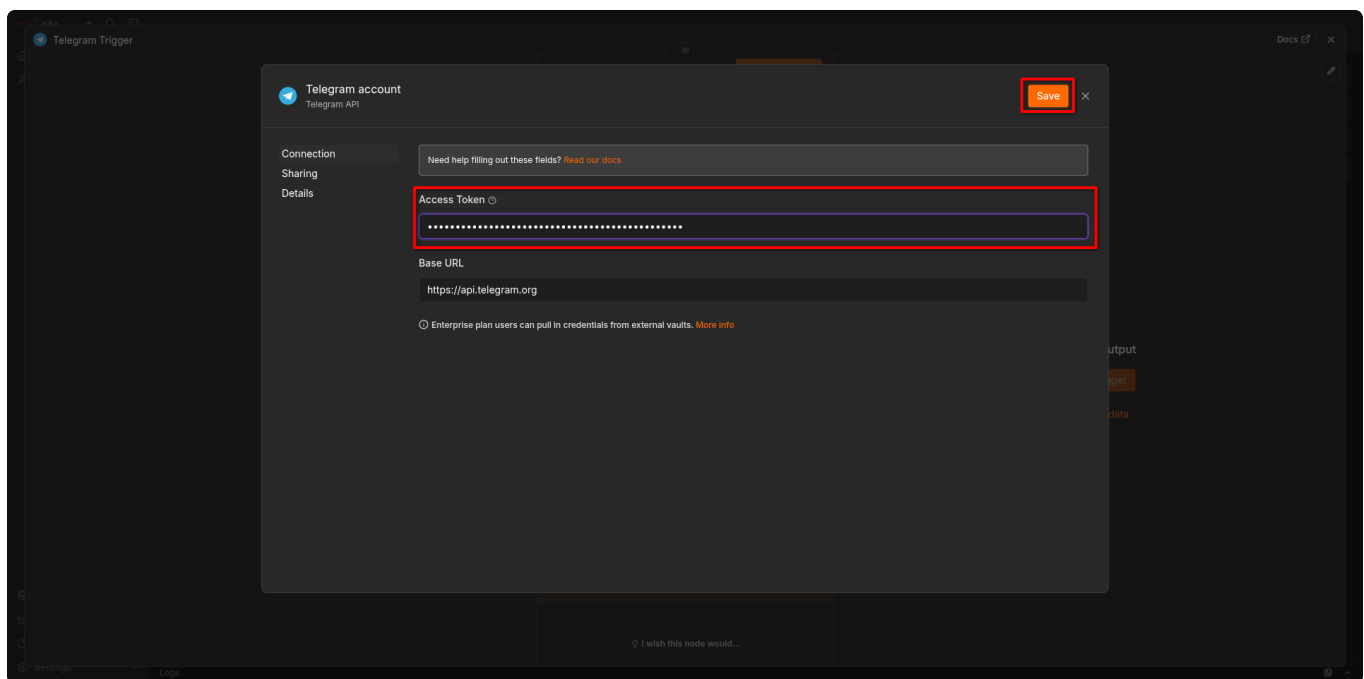
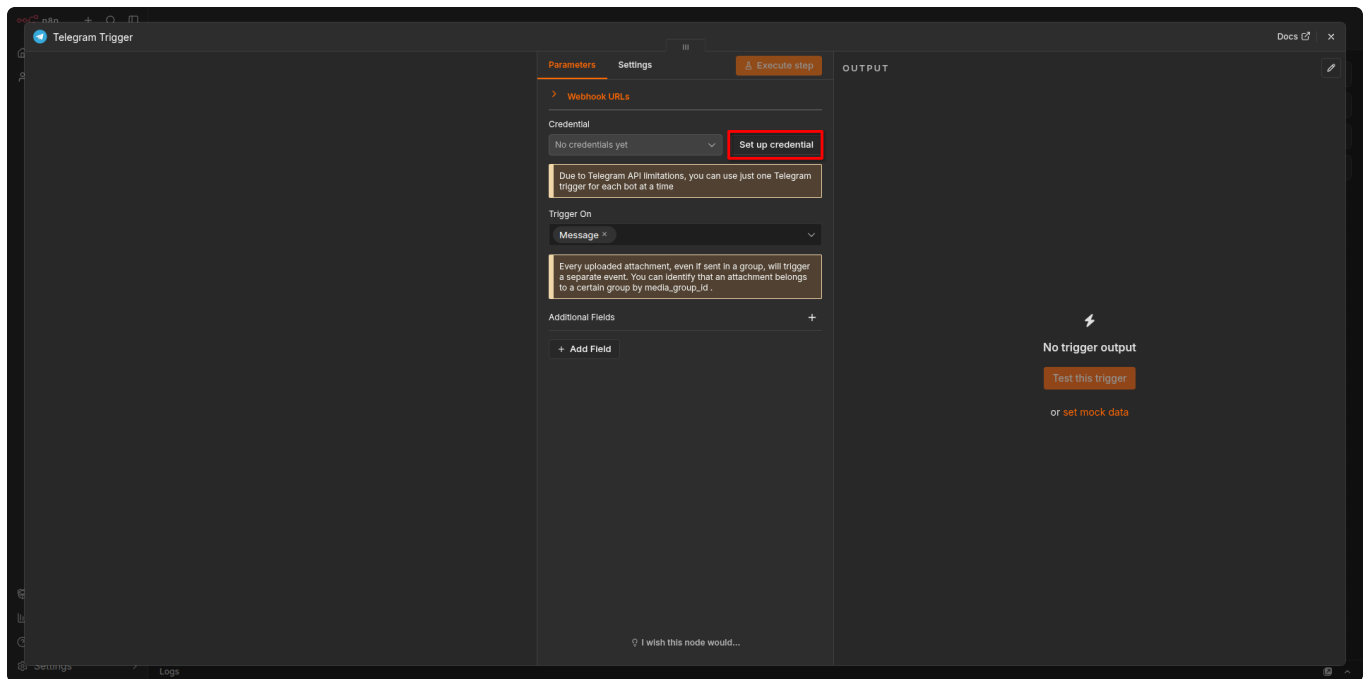


Step 4: Build the n8n Workflow

Node 1: Telegram Trigger

- Node type: **Telegram Trigger**
- Credential: your bot token from Step 1
- Updates: **message**
- This fires every time you message the bot

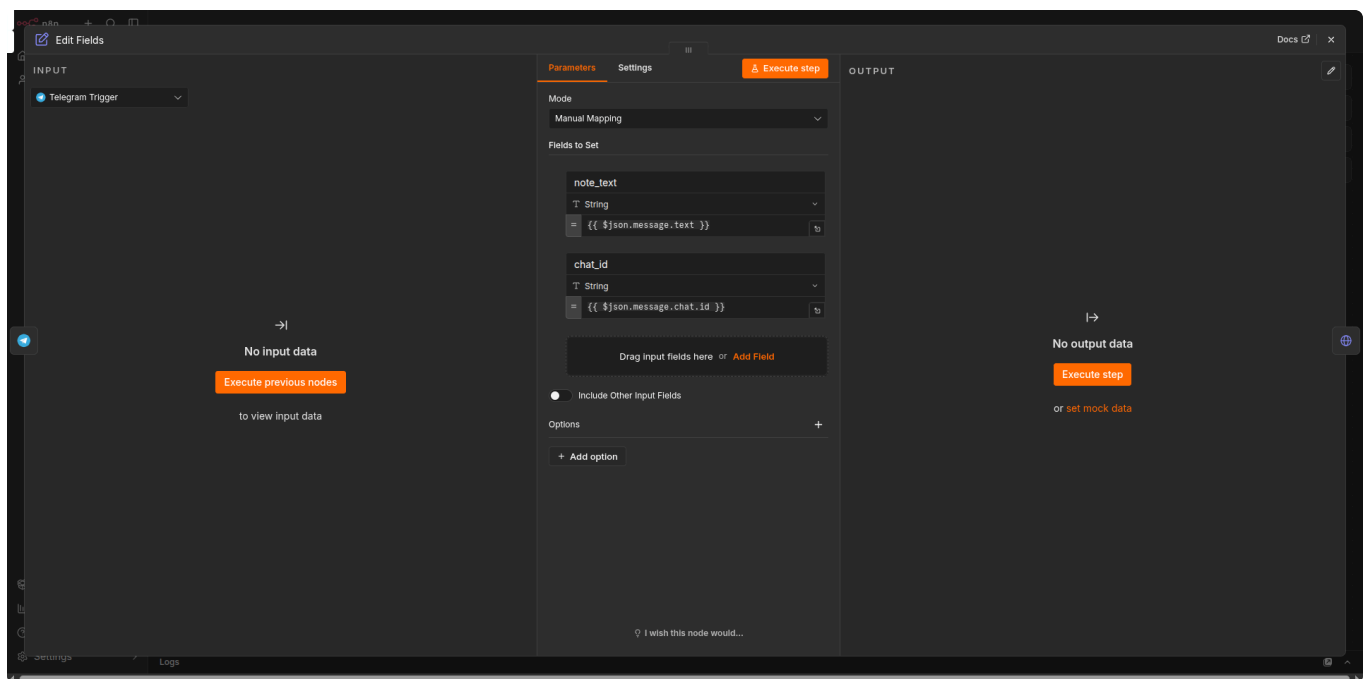
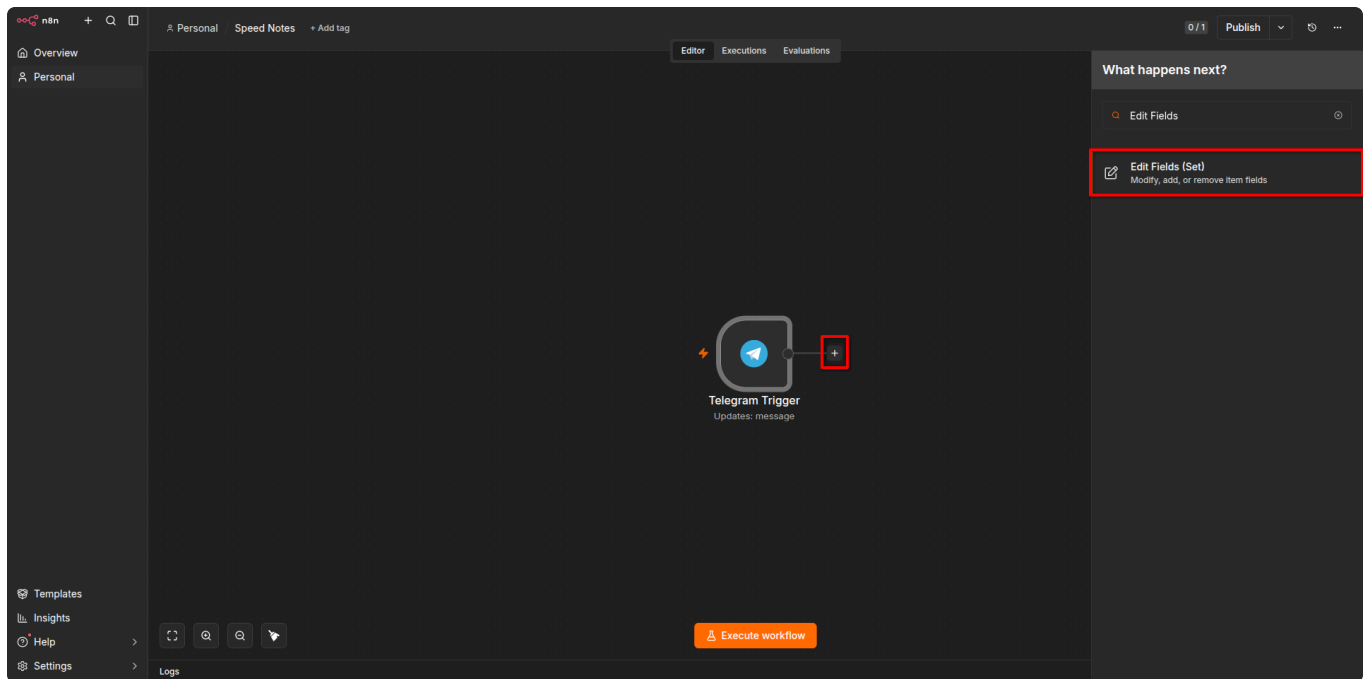




Node 2: Edit Fields (Set)

Extract what you need so later nodes are clean:

- Name: `note_text` → Value: click the expression icon, set to `{{ $json.message.text }}` (no quotes, no `=` prefix — the UI handles that)
- Name: `chat_id` → Value: `{{ $json.message.chat.id }}`

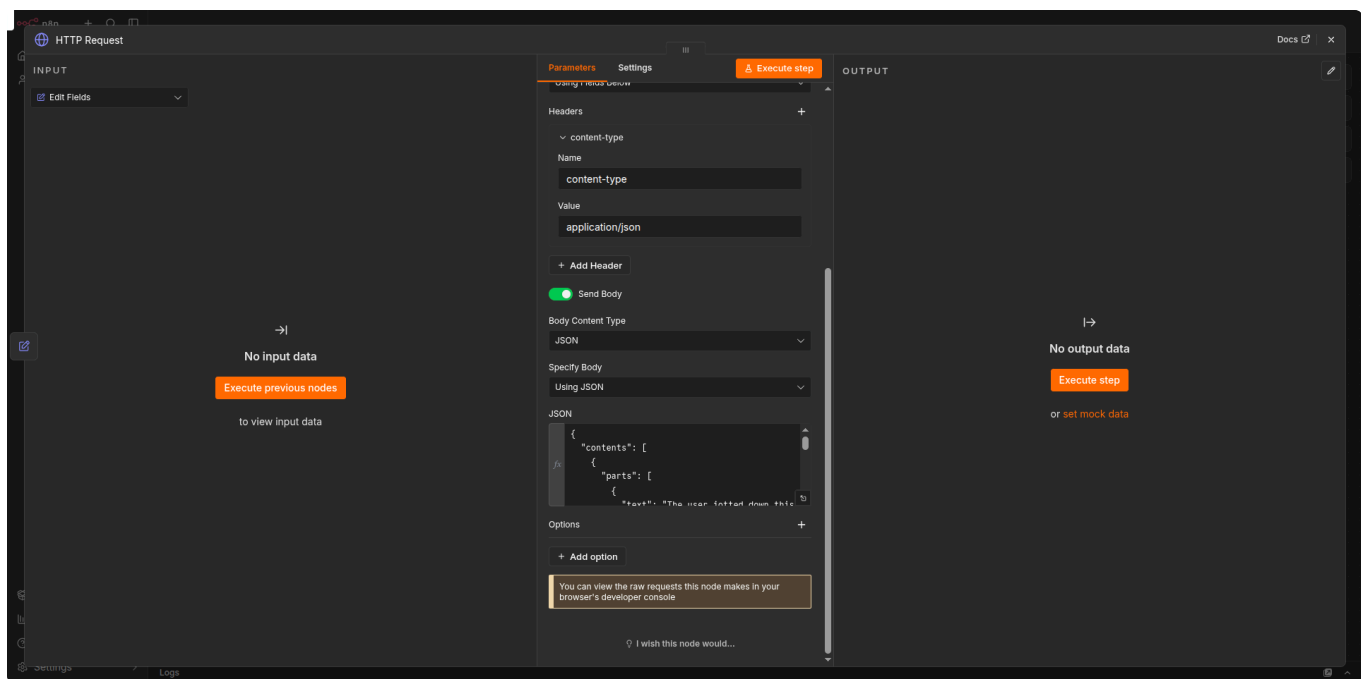
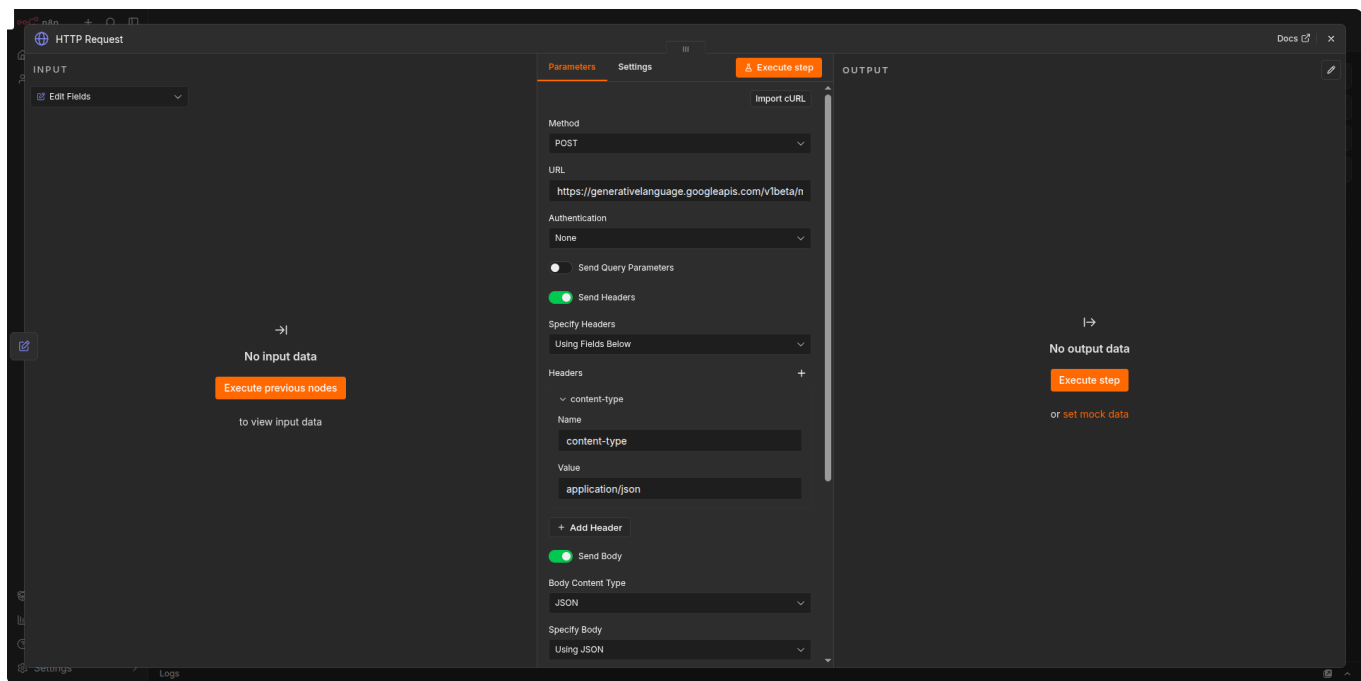


Node 3: HTTP Request (Gemini API call)

- Method: **POST**
- URL: **`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-flash:generateContent?key=YOUR_API_KEY`**
 - Replace **YOUR_API_KEY** with the key from Step 3 (or better, use an n8n **Query Parameter** field for **key** instead of hardcoding it in the URL, so it's not stuck in your workflow history in plain text)
- Headers:
 - **content-type** : **application/json**
- Body (JSON):

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "The user jotted down a quick note, often just 1-3 words, to
look up or remember later: \"{{ $json.note_text }}\"\\n\\nFigure out what this
most likely refers to (a tool, app, game, concept, technology, place, or
anything else) and explain it clearly, the way a knowledgeable friend would if
the user asked \"what is this?\" or \"tell me about this.\"\\n\\nRespond with a
structured note with:\\n1. A clear one-line title (the name of the thing,
cleaned up if there's a typo)\\n2. A 2-4 sentence explanation of what it
actually is, in plain terms\\n3. Practical details worth knowing: what it's
used for, who makes/maintains it, how someone would get started with it, or
similar real-world context. Only mention a connection to cybersecurity,
automation, or software engineering if the topic is genuinely related to those
fields - do not force a professional angle onto unrelated topics like games,
hobbies, or general interests.\\n4. A category, one of: Tool, Concept, Game,
Project Idea, Person/Org, Place, Random\\n\\nRespond ONLY with valid JSON in
this exact format, no markdown fences, no extra text:\\n{\"title\": \" ... \",
\"description\": \" ... \", \"details\": \" ... \", \"category\": \" ... \"}"
        }
      ]
    }
  ]
}
```

The screenshot displays the n8n workflow editor. The main workspace shows a workflow starting with a 'Telegram Trigger' node (labeled 'Updates: message') connected to an 'Edit Fields' node (labeled 'raw'). A red box highlights a '+' button at the end of the workflow chain. On the right side, the 'What happens next?' panel suggests an 'HTTP Request' node, which is also highlighted with a red box. The panel description for the HTTP Request node reads: 'Makes an HTTP request and returns the response data'. The left sidebar contains navigation links for Overview, Personal, Templates, Insights, Help, and Settings. The top bar includes tabs for Editor, Executions, and Evaluations, along with a 'Publish' button and a search icon.



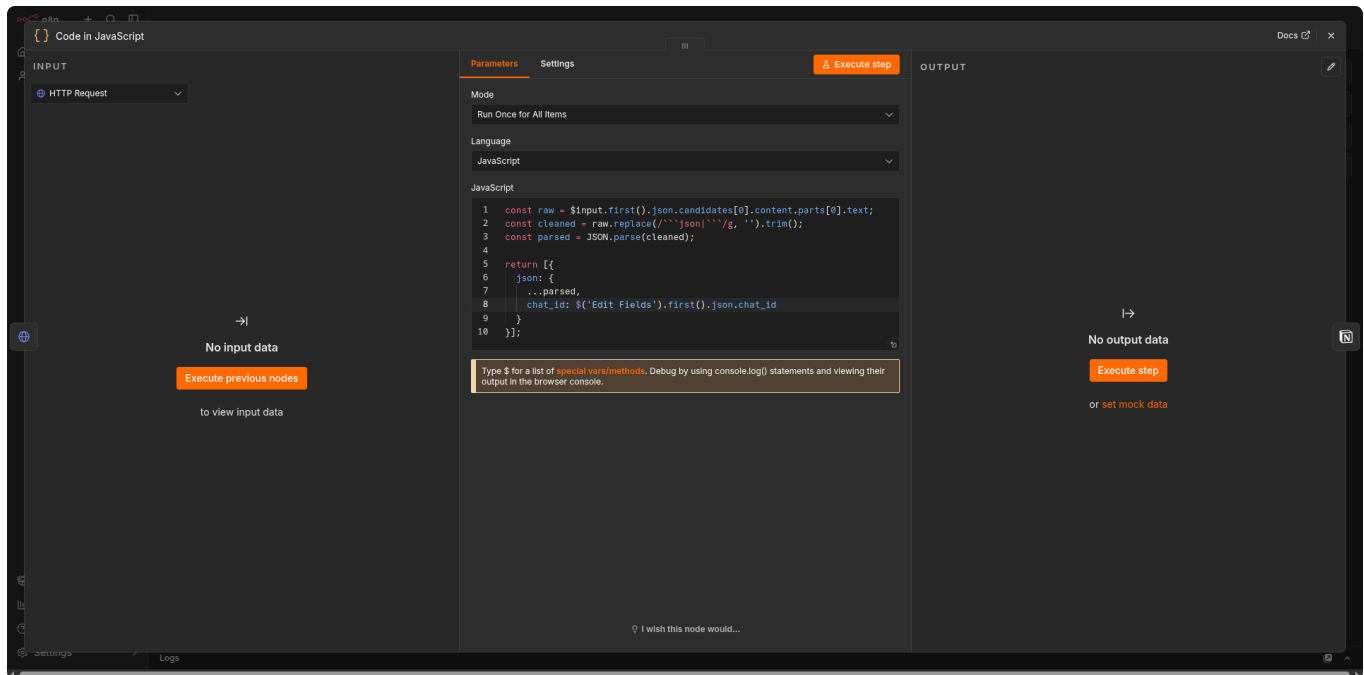
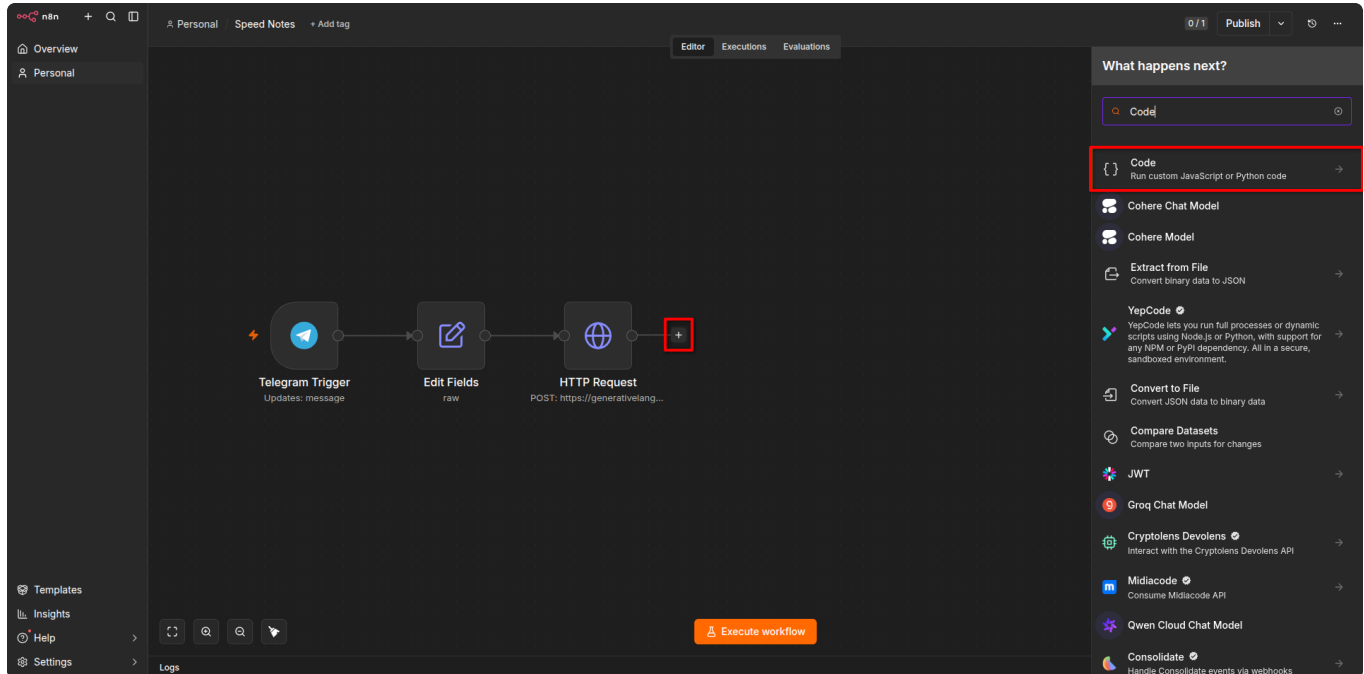
Node 4: Code Node (parse the response)

Gemini's API wraps the reply — you need to pull out and parse the JSON string.

```
const raw = $input.first().json.candidates[0].content.parts[0].text;
const cleaned = raw.replace(/```json|```/g, '').trim();
const parsed = JSON.parse(cleaned);

return [{
  json: {
    ... parsed,
    chat_id: $('Edit Fields').first().json.chat_id
  }
}]
```

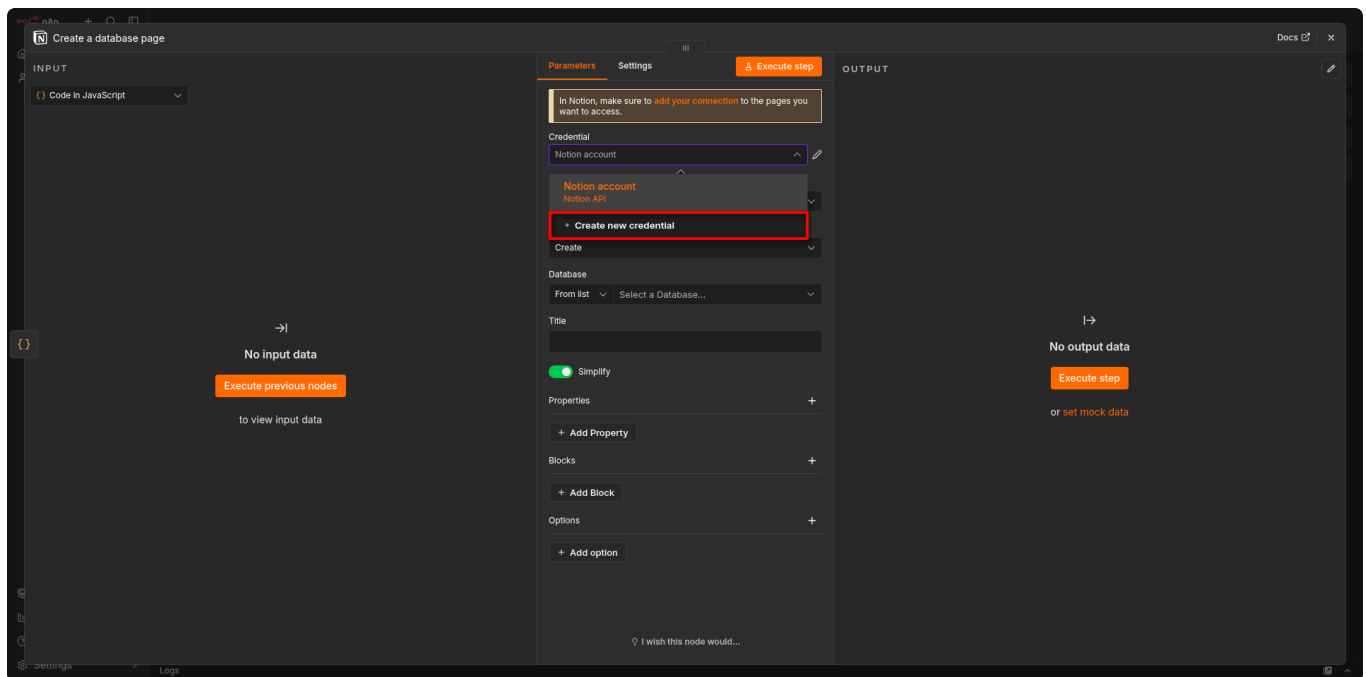
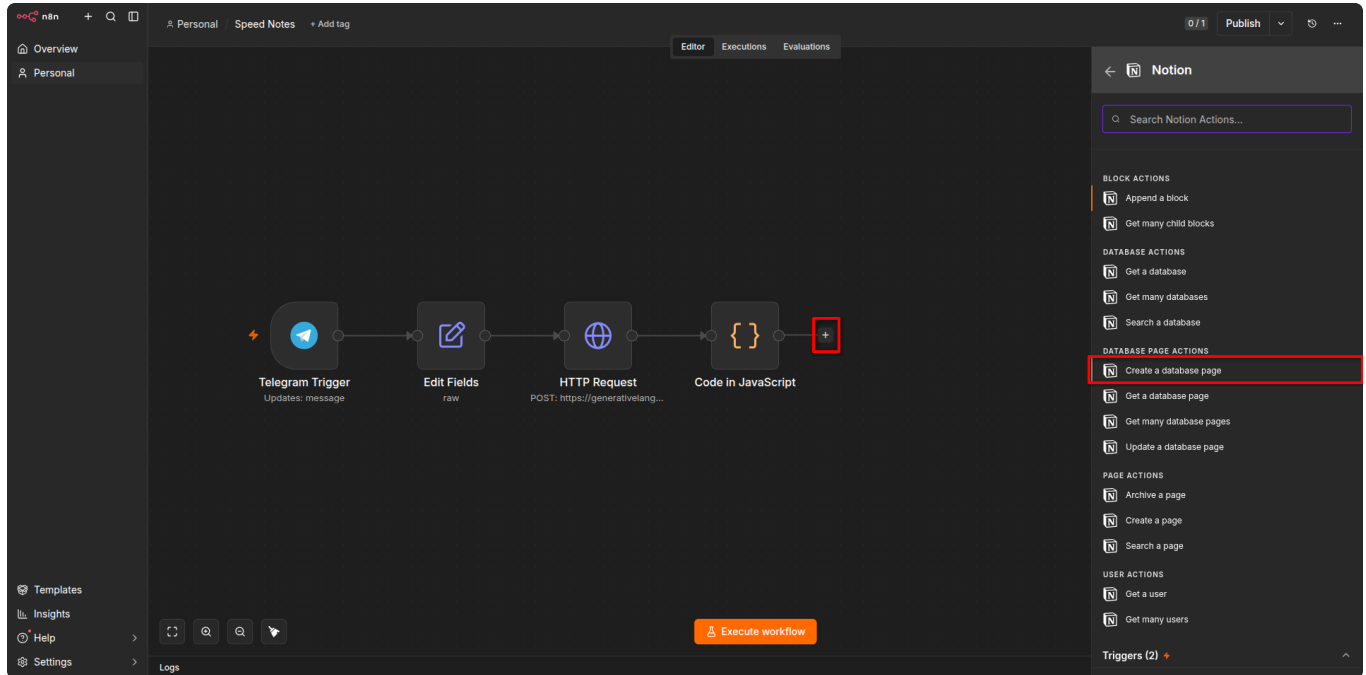
```
}  
}];
```

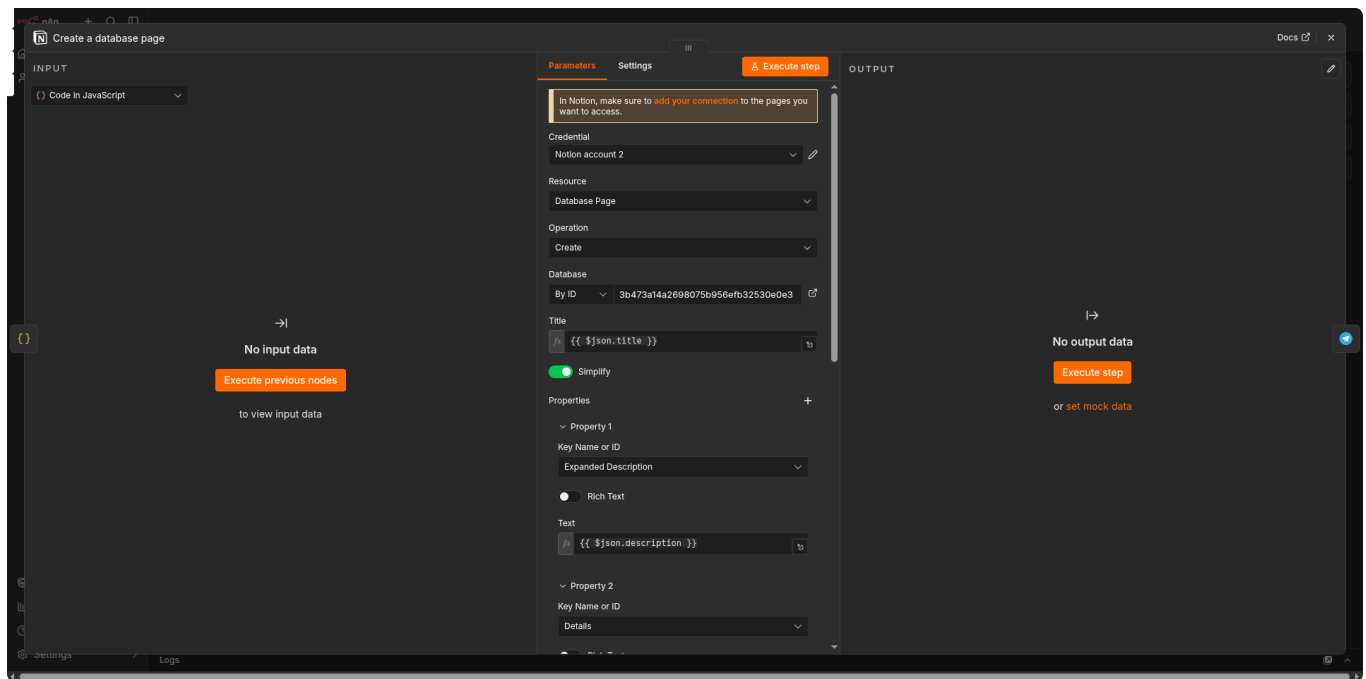
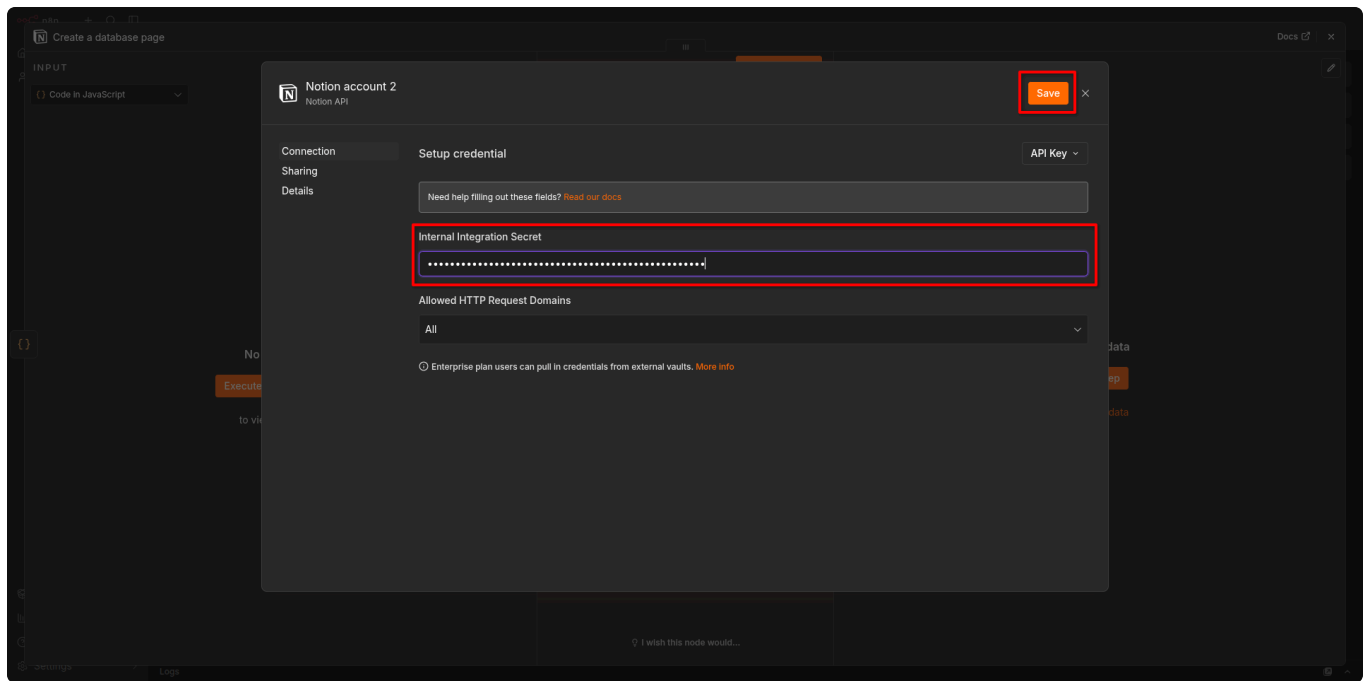


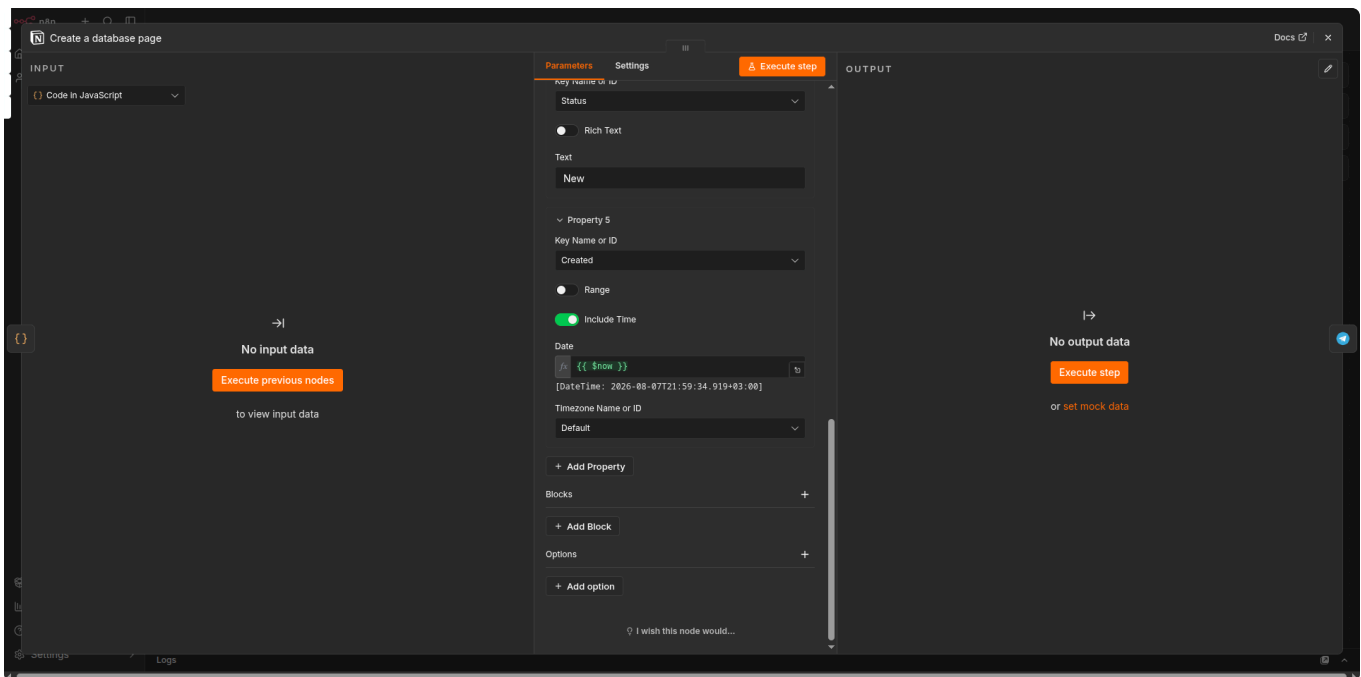
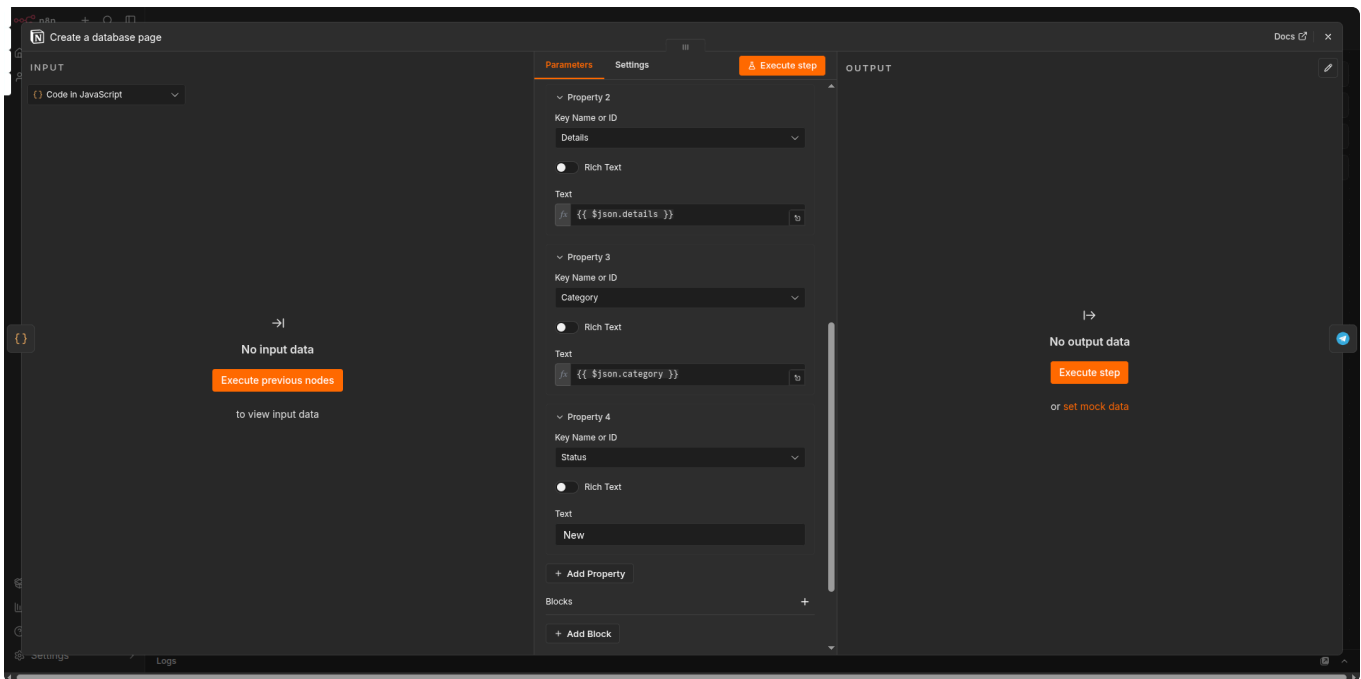
Node 5: Notion Node (Create Page)

- Resource: [Database Page](#)
- Database: select your Idea Vault DB
- Map fields:
 - Title → `{{ $json.title }}`
 - Expanded Description → `{{ $json.description }}`
 - Why Useful → `{{ $json.why_useful }}`

- **Category** → `{{ $json.category }}`
- **Status** → **New**
- **Created** → `{{ $now }}`



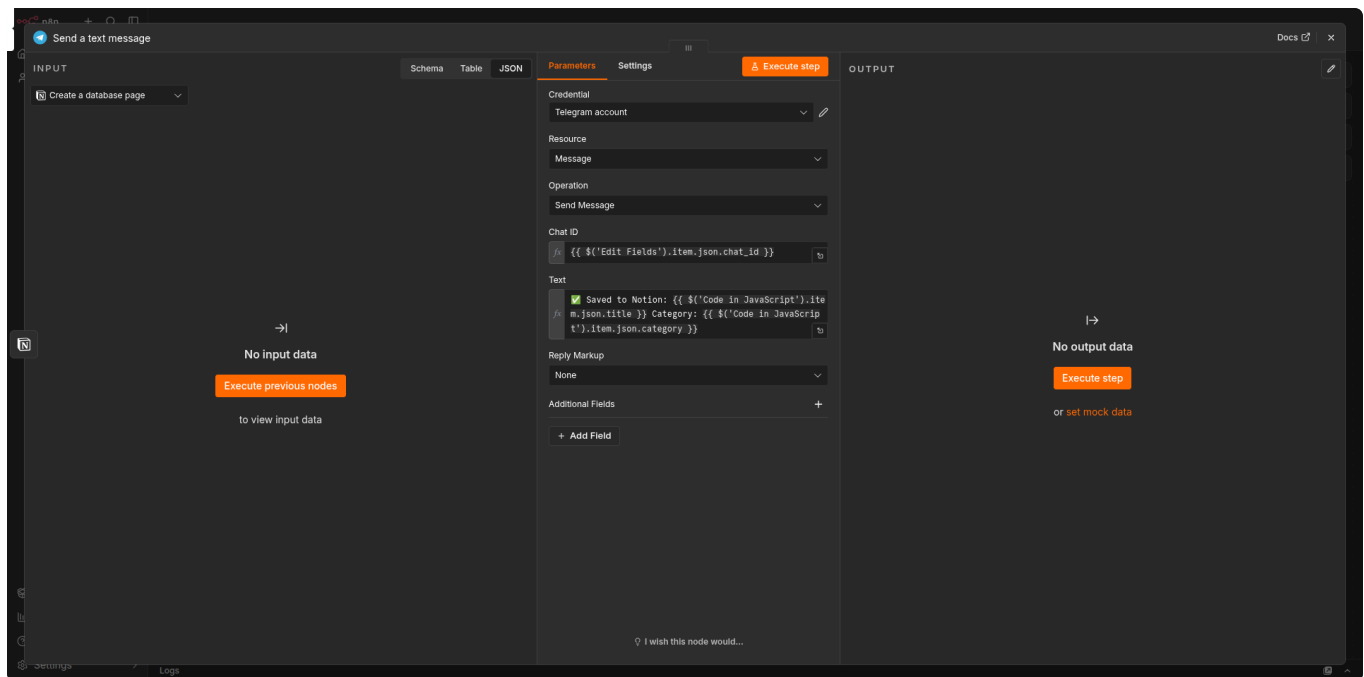
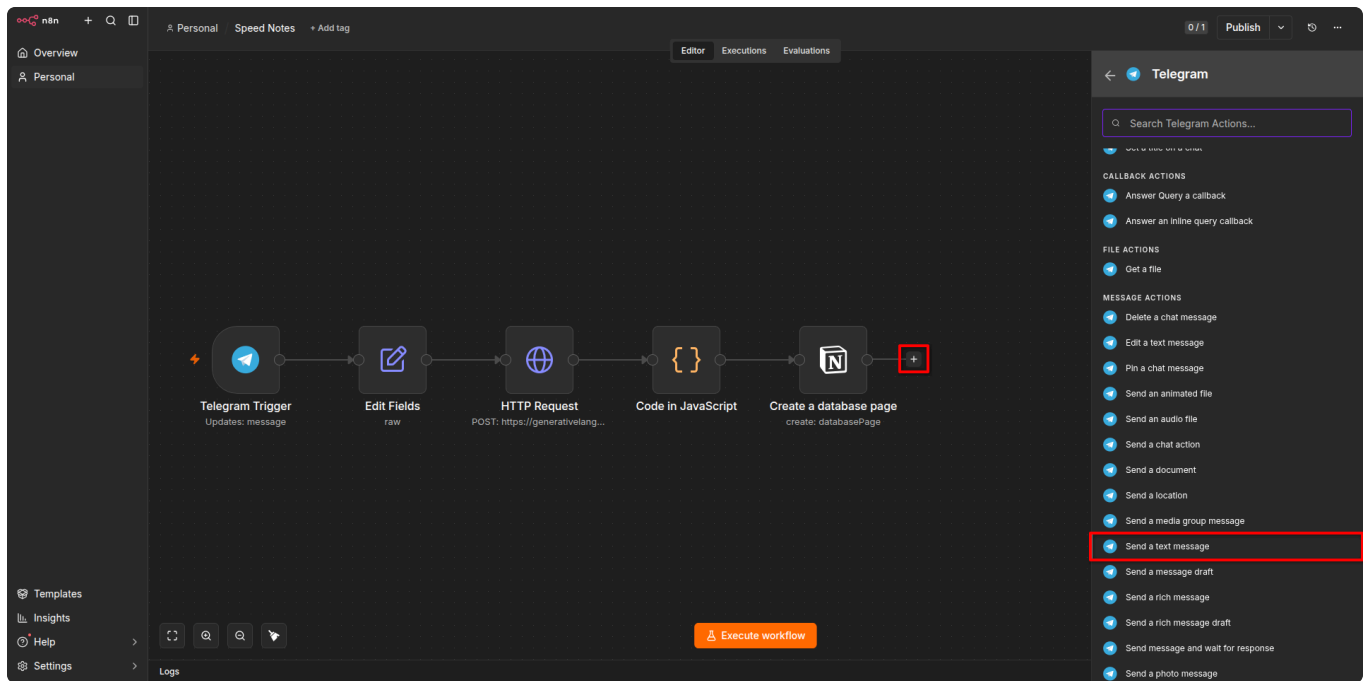




Node 6: Telegram Node (Send Message)

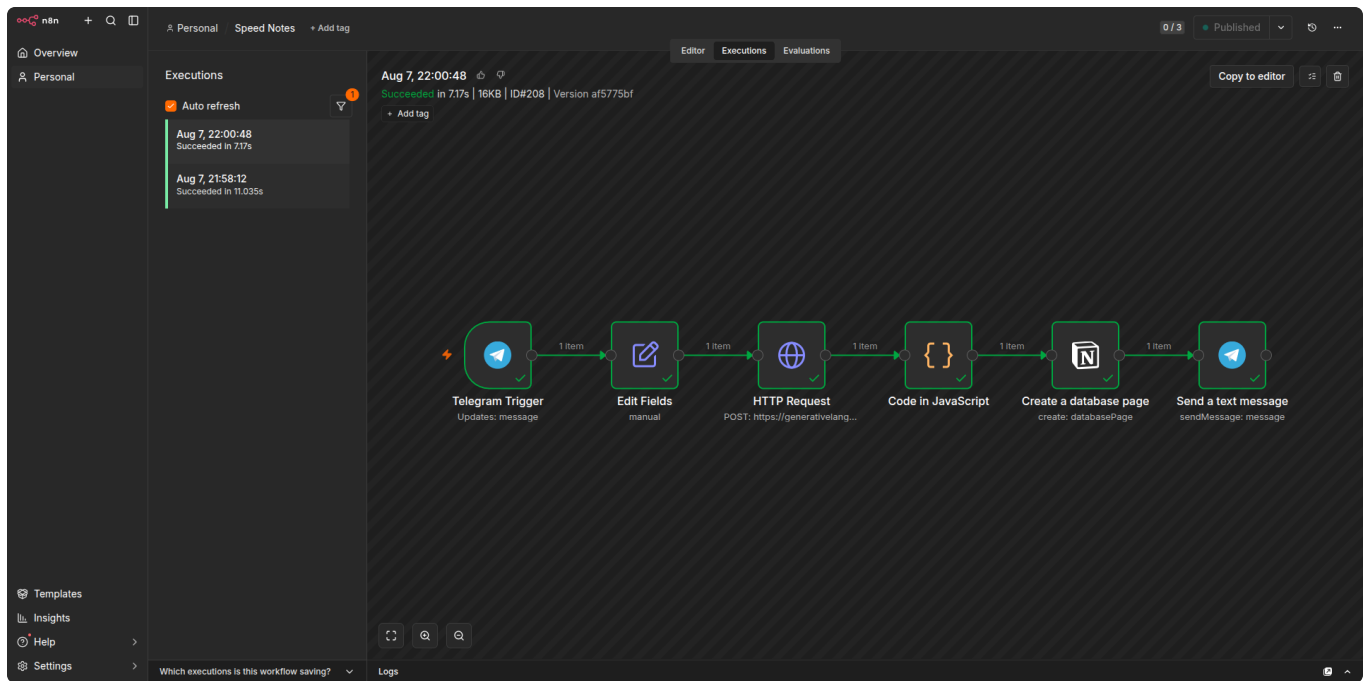
- Chat ID: `{{ $('Edit Fields').item.json.chat_id }}`
- Text:

✓ Saved to Notion: `{{ $('Code in JavaScript').item.json.title }}` Category: `{{ $('Code in JavaScript').item.json.category }}`



Step 5: Connect and Test

1. Wire the nodes in order: Trigger → Edit Fields → HTTP Request → Code → Notion → Telegram
2. Activate the workflow
3. Message your bot something like: `n8n webhook retry logic`
4. Check:
 - Notion gets a new page with a full expanded description
 - Telegram bot replies with a confirmation



Troubleshooting

Problem	Likely Cause
Telegram trigger doesn't fire	Bot needs <code>/start</code> sent to it once first
HTTP Request 401 error	Wrong or missing <code>x-api-key</code> header
Code node JSON.parse fails	Claude added markdown fences or extra text — tighten the prompt, or add more aggressive regex stripping
Notion node fails to map Select field	The <code>Category</code> value returned by Claude doesn't exactly match one of your Notion Select options (case-sensitive) — add a Code step to normalize it, or use a <code>Switch</code> node to map values
Telegram confirmation doesn't send	Chat ID lost between nodes — double check the <code>\$('Edit Fields')</code> reference name in the Code node

Optional Extensions

- Voice notes:** Telegram Trigger also catches voice messages — add a transcription step (Whisper API) before the Claude expansion step

- **Tagging system:** Ask Claude to also return relevant tags, map to a Notion multi-select property
- **Weekly digest:** Separate scheduled workflow that queries Notion for `Status = New` entries and sends you a weekly summary to review/action